

实验报告

系统开发工具基础（第一周）

姓名 甘如嫣

学号 25090032014

日期 2026 年 9 月 1 日

目录

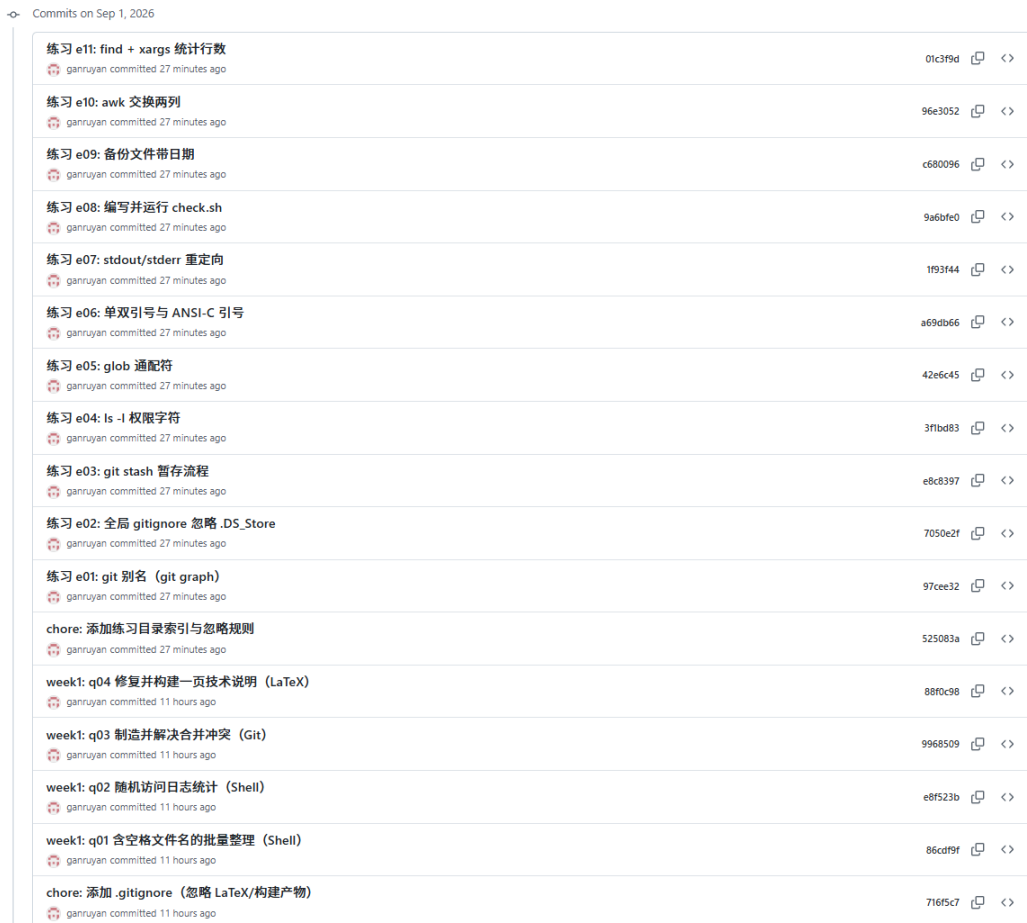
1	GitHub 仓库与提交记录	2
1.1	仓库链接	2
1.2	提交记录	2
2	课上给助教检查的内容与结果	2
2.1	第 1 题：含空格文件名的批量整理	2
2.2	第 2 题：随机访问日志统计	5
2.3	第 3 题：制造、解决并解释一次合并冲突	6
2.4	第 4 题：修复并构建一页技术说明	8
3	课后练习（10 个实例）	10
3.1	实例 1：ls 的 -l 长格式	10
3.2	实例 2：ls -lh 人性化显示文件大小	10
3.3	实例 3：man ls 查看命令手册	11
3.4	实例 4：glob 通配符	12
3.5	实例 5：单引号 / 双引号 / ANSI-C 引号	12
3.6	实例 6：用 vim 编写并运行 check.sh	13
3.7	实例 7：chmod +x 赋予执行权限	13
3.8	实例 8：stdout / stderr 重定向	14
3.9	实例 9：\$? 与 &&、 条件执行	14
3.10	实例 10：带日期的备份文件	15
4	解题感悟	15

1 GitHub 仓库与提交记录

1.1 仓库链接

仓库地址：<https://github.com/ganruyan/system-basics-tools>

1.2 提交记录



Commits on Sep 1, 2026	
练习 e11: find + xargs 统计行数 ganruyan committed 27 minutes ago	01c3f9d <>
练习 e10: awk 交换两列 ganruyan committed 27 minutes ago	96e3052 <>
练习 e09: 备份文件带日期 ganruyan committed 27 minutes ago	c680096 <>
练习 e08: 编写并运行 check.sh ganruyan committed 27 minutes ago	9a6bfe0 <>
练习 e07: stdout/stderr 重定向 ganruyan committed 27 minutes ago	1f93f44 <>
练习 e06: 单双引号与 ANSI-C 引号 ganruyan committed 27 minutes ago	a69db66 <>
练习 e05: glob 通配符 ganruyan committed 27 minutes ago	42e6c45 <>
练习 e04: ls -l 权限字符 ganruyan committed 27 minutes ago	3f1bd83 <>
练习 e03: git stash 暂存流程 ganruyan committed 27 minutes ago	e8c8397 <>
练习 e02: 全局 gitignore 忽略 .DS_Store ganruyan committed 27 minutes ago	7050e2f <>
练习 e01: git 别名 (git graph) ganruyan committed 27 minutes ago	97cee32 <>
chore: 添加练习目录索引与忽略规则 ganruyan committed 27 minutes ago	525083a <>
week1: q04 修复并构建一页技术说明 (LaTeX) ganruyan committed 11 hours ago	88f0c98 <>
week1: q03 制造并解决合并冲突 (Git) ganruyan committed 11 hours ago	9968509 <>
week1: q02 随机访问日志统计 (Shell) ganruyan committed 11 hours ago	e8f523b <>
week1: q01 含空格文件名的批量整理 (Shell) ganruyan committed 11 hours ago	86cd9f9 <>
chore: 添加 .gitignore (忽略 LaTeX/构建产物) ganruyan committed 11 hours ago	716f5c7 <>

图 1: GitHub 提交记录

2 课上给助教检查的内容与结果

本节为课上完成、交由助教检查的四道课堂练习及其结果。

2.1 第 1 题：含空格文件名的批量整理

主题：Shell 基础与文件系统 **建议用时：**12–15 分钟

在当前目录新建 q01，并完全使用命令行完成下列任务。

- 创建 `input/docs` 和 `input/tmp`；创建 `input/docs/notes one.txt`（内容为 `alpha` 和 `beta` 两行）、`input/docs/.secret.txt`（内容为 `hidden`）、`input/tmp/empty.txt`（空文件）以及 `input/run.log`（任意两行日志）。
- 显示 `q01` 的绝对路径，并以长格式列出 `input` 下全部项目，包含隐藏文件。
- 把 `input` 下所有 `.txt` 文件复制到 `work/<25090032014>/`，保留相对目录结构并正确处理名称中的空格。
- 将复制后的目录权限设为 `750`、普通文件权限设为 `640`；生成 `inventory.txt`，列出相对路径和字节数。

结果：

```
gry@gry-VMware-Virtual-Platform:~/q01$ realpath .  
/home/gry/q01  
gry@gry-VMware-Virtual-Platform:~/q01$ ls -la input  
总计 20  
drwxrwxr-x 4 gry gry 4096  8月 31 19:18 .  
drwxrwxr-x 5 gry gry 4096  8月 31 19:28 ..  
drwxrwxr-x 2 gry gry 4096  8月 31 19:17 docs  
-rw-rw-r-- 1 gry gry  20  8月 31 19:48 run.log  
drwxrwxr-x 2 gry gry 4096  8月 31 19:18 tmp  
gry@gry-VMware-Virtual-Platform:~/q01$ ls -laR input  
input:  
总计 20  
drwxrwxr-x 4 gry gry 4096  8月 31 19:18 .  
drwxrwxr-x 5 gry gry 4096  8月 31 19:28 ..  
drwxrwxr-x 2 gry gry 4096  8月 31 19:17 docs  
-rw-rw-r-- 1 gry gry  20  8月 31 19:48 run.log  
drwxrwxr-x 2 gry gry 4096  8月 31 19:18 tmp  
  
input/docs:  
总计 16  
drwxrwxr-x 2 gry gry 4096  8月 31 19:17 .  
drwxrwxr-x 4 gry gry 4096  8月 31 19:18 ..  
-rw-rw-r-- 1 gry gry  11  8月 31 19:48 'notes one.txt'  
-rw-rw-r-- 1 gry gry   7  8月 31 19:48 .secret.txt  
  
input/tmp:  
总计 8  
drwxrwxr-x 2 gry gry 4096  8月 31 19:18 .  
drwxrwxr-x 4 gry gry 4096  8月 31 19:18 ..  
-rw-rw-r-- 1 gry gry   0  8月 31 19:48 empty.txt
```

图 2: 第 1 题 (a) 查看 input 目录结构

```
gry@gry-VMware-Virtual-Platform:~/q01$ ls -laR work/25090032014
work/25090032014:
总计 20
drwxr-x--- 4 gry gry 4096  8月 31 19:51 .
drwxrwxr-x 4 gry gry 4096  8月 31 19:23 ..
drwxr-x--- 2 gry gry 4096  8月 31 19:51 docs
-rw-rw-r-- 1 gry gry   77  8月 31 19:51 inventory.txt
drwxr-x--- 2 gry gry 4096  8月 31 19:51 tmp

work/25090032014/docs:
总计 16
drwxr-x--- 2 gry gry 4096  8月 31 19:51 .
drwxr-x--- 4 gry gry 4096  8月 31 19:51 ..
-rw-r----- 1 gry gry   11  8月 31 19:51 'notes one.txt'
-rw-r----- 1 gry gry    7  8月 31 19:51 .secret.txt

work/25090032014/tmp:
总计 8
drwxr-x--- 2 gry gry 4096  8月 31 19:51 .
drwxr-x--- 4 gry gry 4096  8月 31 19:51 ..
-rw-r----- 1 gry gry    0  8月 31 19:51 empty.txt
gry@gry-VMware-Virtual-Platform:~/q01$ cat work/25090032014/inventory.txt
0  inventory.txt
11 docs/notes one.txt
7  docs/.secret.txt
0  tmp/empty.txt
```

图 3: 第 1 题 (b) 复制结果与 inventory.txt

2.2 第 2 题: 随机访问日志统计

主题: Shell 管道与文本处理 **建议用时:** 12–15 分钟

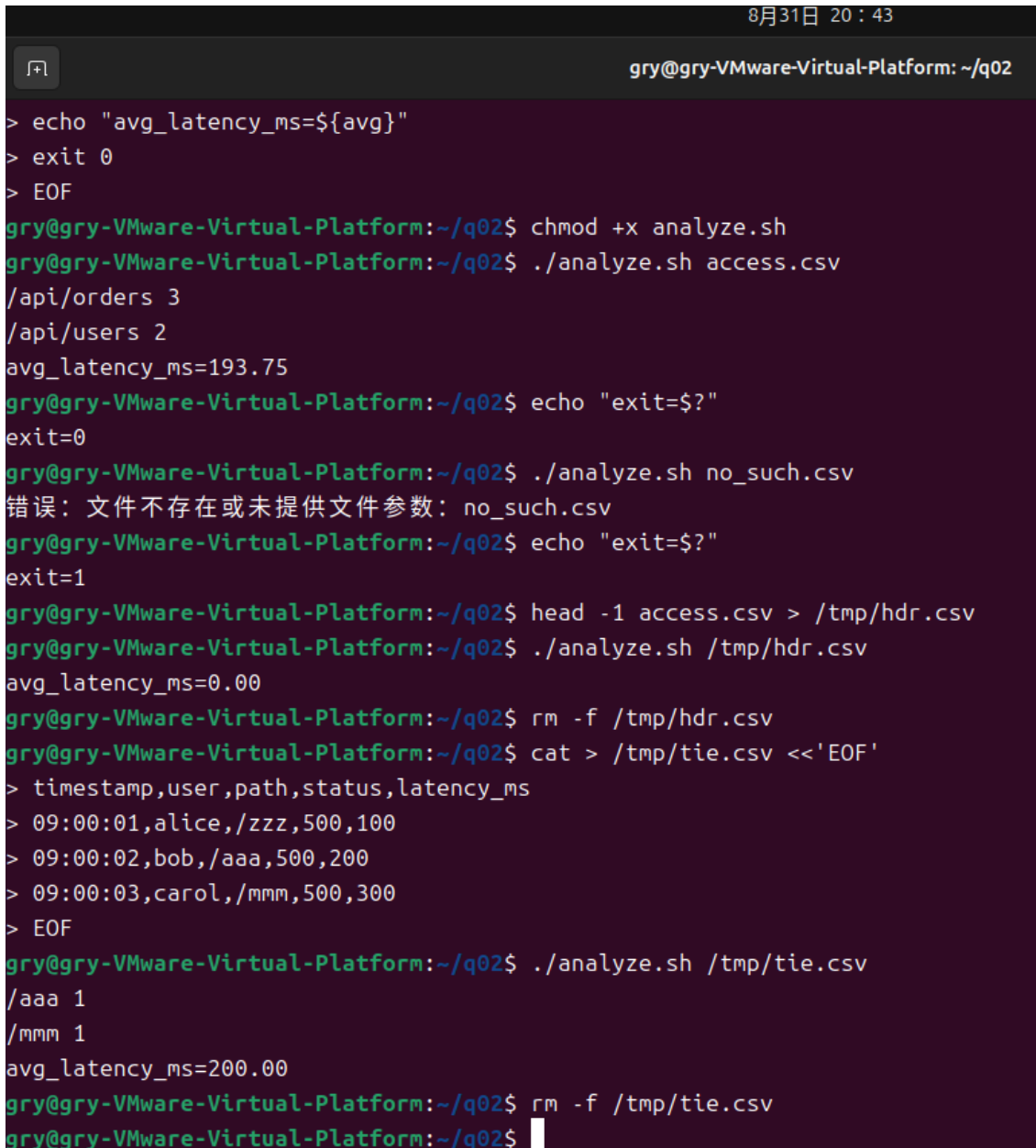
在 q02 中创建 access.csv, 并使用下方给定数据完成日志统计。

```
timestamp,user,path,status,latency_ms
09:00:01,alice,/api/users,200,120
09:00:02,bob,/api/orders,500,310
09:00:03,alice,/api/users,503,180
09:00:04,carol,/login,500,90
09:00:05,bob,/api/orders,502,260
09:00:06,dave,/health,200,20
09:00:07,alice,/api/orders,500,420
09:00:08,carol,/api/users,500,150
```

- 编写 analyze.sh, 接收一个 CSV 文件路径; 文件不存在时将错误写入 stderr 并返回非零退出码。

- 使用 Shell 管道以及 `awk`、`sort`、`head` 等工具，输出 HTTP 5xx 数量最多的前 2 个 path；按次数降序，次数相同时按 path 字典序。
- 输出全部数据行的平均 `latency_ms`，保留两位小数；表头不得计入。
- 分别对 `access.csv` 和一个不存在的文件运行脚本；不得使用 Python、电子表格或手工统计。

结果：



```
8月31日 20:43
gry@gry-VMware-Virtual-Platform: ~/q02
> echo "avg_latency_ms=${avg}"
> exit 0
> EOF
gry@gry-VMware-Virtual-Platform:~/q02$ chmod +x analyze.sh
gry@gry-VMware-Virtual-Platform:~/q02$ ./analyze.sh access.csv
/api/orders 3
/api/users 2
avg_latency_ms=193.75
gry@gry-VMware-Virtual-Platform:~/q02$ echo "exit=$?"
exit=0
gry@gry-VMware-Virtual-Platform:~/q02$ ./analyze.sh no_such.csv
错误：文件不存在或未提供文件参数：no_such.csv
gry@gry-VMware-Virtual-Platform:~/q02$ echo "exit=$?"
exit=1
gry@gry-VMware-Virtual-Platform:~/q02$ head -1 access.csv > /tmp/hdr.csv
gry@gry-VMware-Virtual-Platform:~/q02$ ./analyze.sh /tmp/hdr.csv
avg_latency_ms=0.00
gry@gry-VMware-Virtual-Platform:~/q02$ rm -f /tmp/hdr.csv
gry@gry-VMware-Virtual-Platform:~/q02$ cat > /tmp/tie.csv <<'EOF'
> timestamp,user,path,status,latency_ms
> 09:00:01,alice,/zzz,500,100
> 09:00:02,bob,/aaa,500,200
> 09:00:03,carol,/mmm,500,300
> EOF
gry@gry-VMware-Virtual-Platform:~/q02$ ./analyze.sh /tmp/tie.csv
/aaa 1
/mmm 1
avg_latency_ms=200.00
gry@gry-VMware-Virtual-Platform:~/q02$ rm -f /tmp/tie.csv
gry@gry-VMware-Virtual-Platform:~/q02$
```

图 4: 第 2 题运行结果

2.3 第 3 题：制造、解决并解释一次合并冲突

主题：Version Control and Git 建议用时：12–15 分钟

在 q03 中从零创建一个 Git 仓库，并主动制造、解决一次合并冲突。

- 初始化仓库，在 main 创建 config.txt，内容为 mode=normal，并完成初始提交。
- 创建 feature-a，把内容改为 mode=safe 并提交；从初始 main 创建 feature-b，把内容改为 mode=fast 并提交。
- 回到 main，先合并 feature-a，再合并 feature-b；解决冲突时保留 mode=safe，并另加一行 note=reviewed。
- 确认工作区干净，运行 `git log --all --graph --decorate --oneline` 查看历史。

结果：

```
gry@gry-VMware-Virtual-Platform:~/q03$ git checkout -b feature-a
切换到一个新分支 'feature-a'
gry@gry-VMware-Virtual-Platform:~/q03$ echo "mode=safe" > config.txt
gry@gry-VMware-Virtual-Platform:~/q03$ git add config.txt
gry@gry-VMware-Virtual-Platform:~/q03$ git commit -m "feature-a: change to safe"
[feature-a 9403390] feature-a: change to safe
 1 file changed, 1 insertion(+), 1 deletion(-)
gry@gry-VMware-Virtual-Platform:~/q03$ git checkout main
切换到分支 'main'
gry@gry-VMware-Virtual-Platform:~/q03$ git checkout -b feature-b
切换到一个新分支 'feature-b'
gry@gry-VMware-Virtual-Platform:~/q03$ echo "mode=fast" > config.txt
gry@gry-VMware-Virtual-Platform:~/q03$ git add config.txt
gry@gry-VMware-Virtual-Platform:~/q03$ git commit -m "feature-b: change to fast"
[feature-b b7f2180] feature-b: change to fast
 1 file changed, 1 insertion(+), 1 deletion(-)
gry@gry-VMware-Virtual-Platform:~/q03$ git checkout main
切换到分支 'main'
gry@gry-VMware-Virtual-Platform:~/q03$ git merge feature-a
更新 7f750f5..9403390
Fast-forward
 config.txt | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
gry@gry-VMware-Virtual-Platform:~/q03$ git merge feature-b
自动合并 config.txt
冲突（内容）：合并冲突于 config.txt
自动合并失败，修正冲突然后提交修正的结果。
```

图 5: 第 3 题 (a) 制造合并冲突

```

gry@gry-VMware-Virtual-Platform:~/q03$ git merge feature-a
更新 7f750f5..9403390
Fast-forward
 config.txt | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
gry@gry-VMware-Virtual-Platform:~/q03$ git merge feature-b
自动合并 config.txt
冲突 (内容): 合并冲突于 config.txt
自动合并失败, 修正冲突然后提交修正的结果。
gry@gry-VMware-Virtual-Platform:~/q03$ mode=safe
gry@gry-VMware-Virtual-Platform:~/q03$ note=reviewed
gry@gry-VMware-Virtual-Platform:~/q03$ git add config.txt
gry@gry-VMware-Virtual-Platform:~/q03$ git commit -m "merge feature-b, resolved conflict: keep safe, add note"
[main 1631536] merge feature-b, resolved conflict: keep safe, add note
gry@gry-VMware-Virtual-Platform:~/q03$ git status
位于分支 main
未跟踪的文件:
  (使用 "git add <文件>..." 以包含要提交的内容)
    a.txt

提交为空, 但是存在尚未跟踪的文件 (使用 "git add" 建立跟踪)
gry@gry-VMware-Virtual-Platform:~/q03$ git log --all --graph --decorate --oneline
* 1631536 (HEAD -> main) merge feature-b, resolved conflict: keep safe, add note
|
| * b7f2180 (feature-b) feature-b: change to fast
| * | 9403390 (feature-a) feature-a: change to safe
|/
* 7f750f5 init: mode=normal
gry@gry-VMware-Virtual-Platform:~/q03$

```

图 6: 第 3 题 (b) 解决冲突并查看历史

2.4 第 4 题: 修复并构建一页技术说明

主题: LaTeX 文档编辑 **建议用时:** 12–15 分钟

将下方模板保存为 q04/report.tex, 补全内容并从命令行构建 PDF。

```

\documentclass{article}
\usepackage{amsmath}
\begin{document}
\title{Tool Report}
\author{姓名—学号}
\maketitle
\section{Result}
% 在此加入公式、表格及正文引用
\end{document}

```

- 加入一个带编号和 label 的公式, 并在正文中使用 ref 或 eqref 引用。
- 加入一个 2 行 2 列表格, 设置 caption 和 label, 并在正文中引用该表。
- 使用 latexmk -pdf -halt-on-error 或 tectonic 生成 report.pdf。
- 确认 PDF 能打开, 且交叉引用不显示“??”。

结果：

```

-----
This is pdfTeX, Version 3.141592653-2.6-1.40.25 (TeX Live 2023/Debian) (preloaded format=pdflatex)
 restricted \write18 enabled.
entering extended mode
(./report.tex
LaTeX2e <2023-11-01> patch level 1
L3 programming layer <2024-01-22>
(/usr/share/texlive/texmf-dist/tex/latex/base/article.cls
Document Class: article 2023/05/17 v1.4n Standard LaTeX document class
(/usr/share/texlive/texmf-dist/tex/latex/base/size10.clo))
(/usr/share/texlive/texmf-dist/tex/latex/amsmath/amsmath.sty
For additional information on amsmath, use the '?' option.
(/usr/share/texlive/texmf-dist/tex/latex/amsmath/amstext.sty
(/usr/share/texlive/texmf-dist/tex/latex/amsmath/amsgen.sty))
(/usr/share/texlive/texmf-dist/tex/latex/amsmath/amsbsy.sty)
(/usr/share/texlive/texmf-dist/tex/latex/amsmath/amsopn.sty))
(/usr/share/texlive/texmf-dist/tex/latex/l3backend/l3backend-pdfTeX.def)
(./report.aux) [1{/var/lib/texmf/fonts/map/pdftex/updmap/pdftex.map}]
(./report.aux) </usr/share/texlive/texmf-dist/fonts/type1/public/amsfonts/cm/cm
mbx12.pfb></usr/share/texlive/texmf-dist/fonts/type1/public/amsfonts/cm/cm
mi10.pfb></usr/share/texlive/texmf-dist/fonts/type1/public/amsfonts/cm/cm
r10.pfb></usr/share/texlive/texmf-dist/fonts/type1/public/amsfonts/cm/cm
r12.pfb></usr/share/texlive/texmf-dist/fonts/type1/public/amsfonts/cm/cm
r17.pfb>
Output written on report.pdf (1 page, 56180 bytes).
Transcript written on report.log.
Latexmk: Getting log file 'report.log'
Latexmk: Examining 'report.flx'
Latexmk: Examining 'report.log'
Latexmk: Log file says output to 'report.pdf'
Latexmk: All targets (report.pdf) are up-to-date

```

图 7: 第 4 题 (a) 从命令行编译 PDF

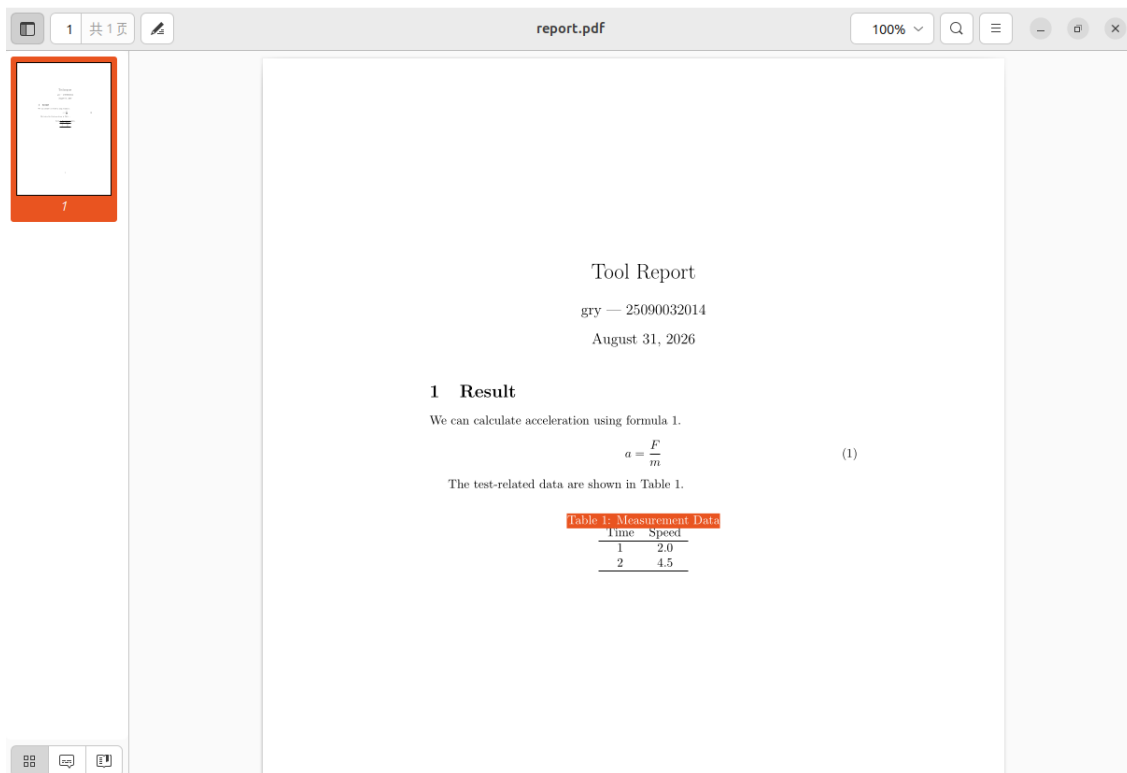


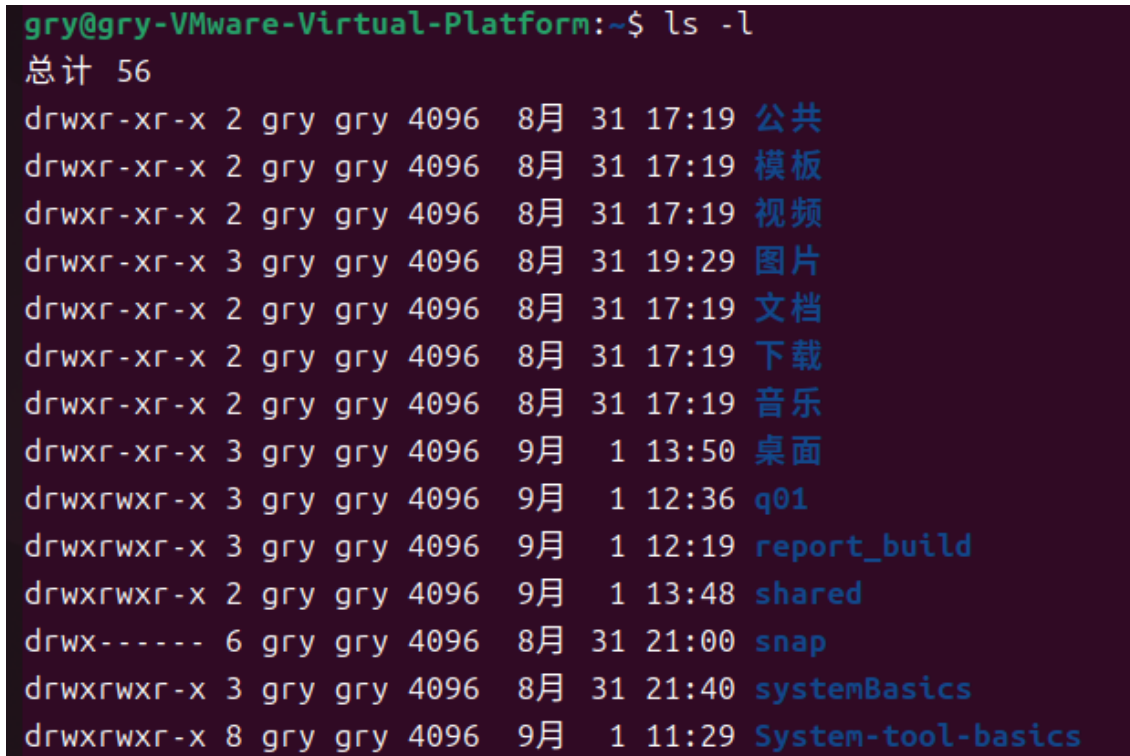
图 8: 第 4 题 (b) 生成的 PDF 与交叉引用

3 课后练习（10 个实例）

课后练习共完成 10 个实例，全部在 Ubuntu 虚拟机的终端中实际操作完成。

3.1 实例 1: ls 的 -l 长格式

题目：运行 `ls -l` 查看当前目录的长格式列表，观察输出每一行前 10 个字符的含义——第 1 个字符是文件类型（d 表示目录），随后 9 个字符是 owner / group / other 三组的 `rwX` 权限位。



```
gry@gry-VMware-Virtual-Platform:~$ ls -l
总计 56
drwxr-xr-x 2 gry gry 4096  8月 31 17:19 公共
drwxr-xr-x 2 gry gry 4096  8月 31 17:19 模板
drwxr-xr-x 2 gry gry 4096  8月 31 17:19 视频
drwxr-xr-x 3 gry gry 4096  8月 31 19:29 图片
drwxr-xr-x 2 gry gry 4096  8月 31 17:19 文档
drwxr-xr-x 2 gry gry 4096  8月 31 17:19 下载
drwxr-xr-x 2 gry gry 4096  8月 31 17:19 音乐
drwxr-xr-x 3 gry gry 4096  9月  1 13:50 桌面
drwxrwxr-x 3 gry gry 4096  9月  1 12:36 q01
drwxrwxr-x 3 gry gry 4096  9月  1 12:19 report_build
drwxrwxr-x 2 gry gry 4096  9月  1 13:48 shared
drwx----- 6 gry gry 4096  8月 31 21:00 snap
drwxrwxr-x 3 gry gry 4096  8月 31 21:40 systemBasics
drwxrwxr-x 8 gry gry 4096  9月  1 11:29 System-tool-basics
```

图 9: 实例 1: `ls -l` 长格式输出

3.2 实例 2: ls -lh 人性化显示文件大小

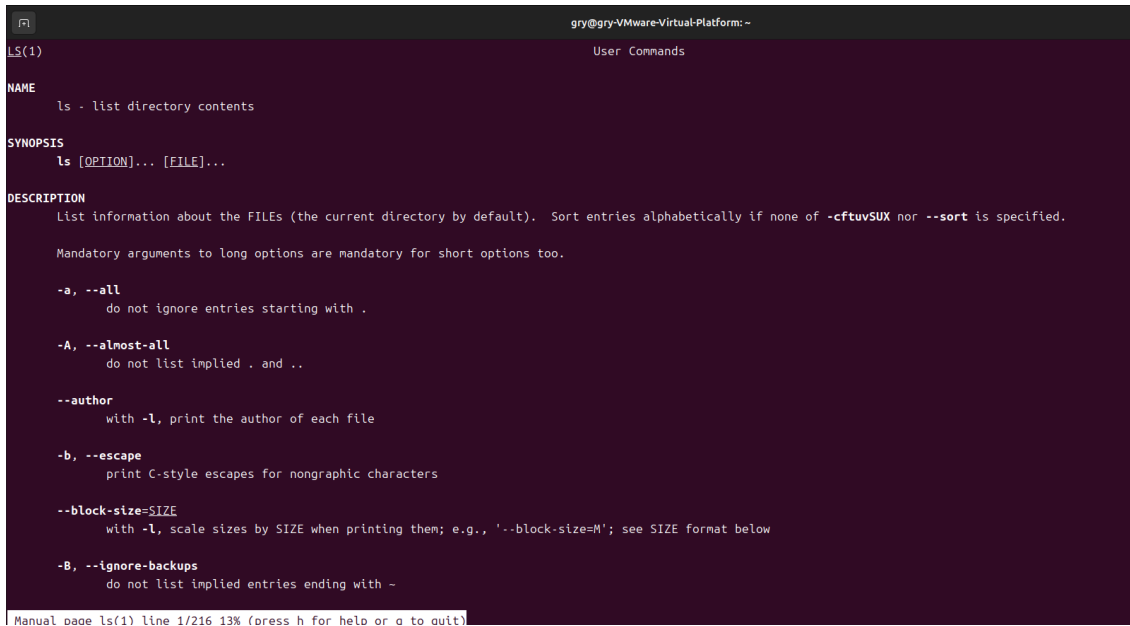
题目：运行 `ls -lh /`，用 K/M/G 等单位人性化显示文件大小，并观察根目录下各目录的权限位。

```
gry@gry-VMware-Virtual-Platform:~$ ls -lh /
总计 3.6G
lrwxrwxrwx   1 root root    7 4月 22  2024 bin -> usr/bin
drwxr-xr-x   2 root root 4.0K 2月 26  2024 bin.usr-is-merged
drwxr-xr-x   3 root root 4.0K 8月 31 17:13 boot
dr-xr-xr-x   2 root root 4.0K 2月 10  2026 cdrom
drwxr-xr-x  19 root root 4.2K 9月  1 13:46 dev
drwxr-xr-x 145 root root 12K  9月  1 12:08 etc
drwxr-xr-x   3 root root 4.0K 8月 31 16:52 home
lrwxrwxrwx   1 root root    9 4月 22  2024 lib -> usr/lib
lrwxrwxrwx   1 root root    9 4月 22  2024 lib64 -> usr/lib64
drwxr-xr-x   2 root root 4.0K 4月  8  2024 lib.usr-is-merged
drwx-----  2 root root 16K 8月 31 16:39 lost+found
drwxr-xr-x   3 root root 4.0K 8月 31 18:28 media
drwxr-xr-x   3 root root 4.0K 8月 31 18:48 mnt
drwxr-xr-x   2 root root 4.0K 2月 10  2026 opt
dr-xr-xr-x 412 root root    0 9月  1 13:46 proc
drwx-----  6 root root 4.0K 8月 31 20:20 root
drwxr-xr-x  37 root root 940  9月  1 13:50 run
lrwxrwxrwx   1 root root    8 4月 22  2024/sbin -> usr/sbin
drwxr-xr-x   2 root root 4.0K 3月 31  2024/sbin.usr-is-merged
drwxr-xr-x  15 root root 4.0K 9月  1 00:18 snap
drwxr-xr-x   2 root root 4.0K 2月 10  2026 srv
-rw-----  1 root root 3.6G 8月 31 16:52 swap.img
dr-xr-xr-x  13 root root    0 9月  1 13:50 sys
drwxrwxrwt  19 root root 4.0K 9月  1 13:48 tmp
drwxr-xr-x  12 root root 4.0K 2月 10  2026 usr
drwxr-xr-x  14 root root 4.0K 8月 31 17:18 var
```

图 10: 实例 2: `ls -lh /` 的输出

3.3 实例 3: `man ls` 查看命令手册

题目: 运行 `man ls` 查看 `ls` 的完整选项说明, 了解 `-a`、`-l`、`-h` 等常用选项的含义, 按 `q` 退出手册。



```
gry@gry-VMware-Virtual-Platform: ~
LS(1)
NAME
    ls - list directory contents

SYNOPSIS
    ls [OPTION]... [FILE]...

DESCRIPTION
    List information about the FILES (the current directory by default). Sort entries alphabetically if none of -cftuvSUX nor --sort is specified.

    Mandatory arguments to long options are mandatory for short options too.

    -a, --all
        do not ignore entries starting with .

    -A, --almost-all
        do not list implied . and ..

    --author
        with -l, print the author of each file

    -b, --escape
        print C-style escapes for nongraphic characters

    --block-size=SIZE
        with -l, scale sizes by SIZE when printing them; e.g., '--block-size=M'; see SIZE format below

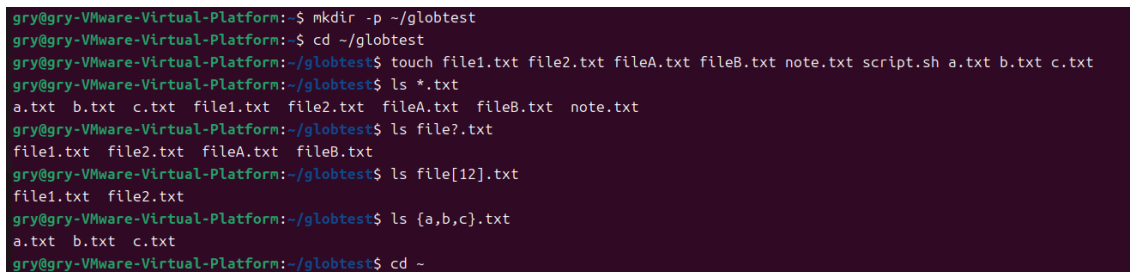
    -B, --ignore-backups
        do not list implied entries ending with ~

Manual page ls(1) line 1/216 13% (press h for help or q to quit)
```

图 11: 实例 3: man ls 手册页

3.4 实例 4: glob 通配符

题目: 在 globtest 目录中创建 9 个测试文件 file1.txt file2.txt fileA.txt fileB.txt note.txt script.sh a.txt b.txt c.txt, 分别用 * ? [12] {a,b,c} 等模式匹配。



```
gry@gry-VMware-Virtual-Platform:~$ mkdir -p ~/globtest
gry@gry-VMware-Virtual-Platform:~$ cd ~/globtest
gry@gry-VMware-Virtual-Platform:~/globtest$ touch file1.txt file2.txt fileA.txt fileB.txt note.txt script.sh a.txt b.txt c.txt
gry@gry-VMware-Virtual-Platform:~/globtest$ ls *.txt
a.txt b.txt c.txt file1.txt file2.txt fileA.txt fileB.txt note.txt
gry@gry-VMware-Virtual-Platform:~/globtest$ ls file?.txt
file1.txt file2.txt fileA.txt fileB.txt
gry@gry-VMware-Virtual-Platform:~/globtest$ ls file[12].txt
file1.txt file2.txt
gry@gry-VMware-Virtual-Platform:~/globtest$ ls {a,b,c}.txt
a.txt b.txt c.txt
gry@gry-VMware-Virtual-Platform:~/globtest$ cd ~
```

图 12: 实例 4: 各通配符的匹配结果

3.5 实例 5: 单引号 / 双引号 / ANSI-C 引号

题目: 用 echo 对比单引号 (\$HOME 不展开)、双引号 (\$HOME 展开为实际路径)、ANSI-C 引号 \$'...' (解析 \n 等转义) 三者的区别。

```
gry@gry-VMware-Virtual-Platform:~$ echo '单引号：$HOME 不展开'
单引号：$HOME 不展开
gry@gry-VMware-Virtual-Platform:~$ echo "双引号：$HOME 会展开"
双引号：/home/gry 会展开
gry@gry-VMware-Virtual-Platform:~$ printf '$\n!\n'
$
!
gry@gry-VMware-Virtual-Platform:~$ echo '$$\n!'
$
!
gry@gry-VMware-Virtual-Platform:~$ echo '$'第一行\n第二行'
第一行
第二行
```

图 13: 实例 5: 三种引号的输出对比

3.6 实例 6: 用 vim 编写并运行 check.sh

题目: 用 vim 编写 check.sh, 用 [-f ...] (即 test -f) 判断传入的文件是否存在, 并分别对不存在的 somefile 和存在的 /etc/passwd 运行脚本。

```
gry@gry-VMware-Virtual-Platform:~$ mkdir -p ~/scripts
gry@gry-VMware-Virtual-Platform:~$ cd ~/scripts
gry@gry-VMware-Virtual-Platform:~/scripts$ vim check.sh
gry@gry-VMware-Virtual-Platform:~/scripts$ vim check.sh
gry@gry-VMware-Virtual-Platform:~/scripts$ bash check.sh somefile
文件不存在: somefile
gry@gry-VMware-Virtual-Platform:~/scripts$ bash check.sh /etc/passwd
文件存在: /etc/passwd
gry@gry-VMware-Virtual-Platform:~/scripts$
```

图 14: 实例 6: check.sh 的运行结果

3.7 实例 7: chmod +x 赋予执行权限

题目: 先不带执行权限运行 ./check.sh, 观察“权限不够”的报错, 再用 chmod +x 赋予执行权限后重新运行, 对比前后权限位的变化。

```

gry@gry-VMware-Virtual-Platform:~/scripts$ cd ~/scripts
gry@gry-VMware-Virtual-Platform:~/scripts$ ls -l check.sh
-rw-rw-r-- 1 gry gry 95  9月  1 14:12 check.sh
gry@gry-VMware-Virtual-Platform:~/scripts$ ./check.sh somefile
bash: ./check.sh: 权限不够
gry@gry-VMware-Virtual-Platform:~/scripts$ ./check.sh somefile
bash: ./check.sh: 权限不够
gry@gry-VMware-Virtual-Platform:~/scripts$ ls -l check.sh
-rw-rw-r-- 1 gry gry 95  9月  1 14:12 check.sh
gry@gry-VMware-Virtual-Platform:~/scripts$ ./check.sh somefile
bash: ./check.sh: 权限不够
gry@gry-VMware-Virtual-Platform:~/scripts$

```

图 15: 实例 7: chmod +x 前后对比

3.8 实例 8: stdout / stderr 重定向

题目: 运行 `ls /nonexistent /tmp` (既有报错又有正常输出), 分别用 `1>out.txt` `2>err.txt` 把 stdout 与 stderr 重定向到不同文件, 再用 `cat` 查看两个文件的内容, 理解标准输出与标准错误的区别。

```

gry@gry-VMware-Virtual-Platform:~/scripts$ ls /nonexistent /tmp
ls: 无法访问 '/nonexistent': 没有那个文件或目录
/tmp:
gdn3-config-err-KY4F8a
snap-private-tmp
systemd-private-cba9c77d21c6474391b8b8b746d3515e-colord.service-ixefik
systemd-private-cba9c77d21c6474391b8b8b746d3515e-fwupd.service-dLidwh
systemd-private-cba9c77d21c6474391b8b8b746d3515e-ModemManager.service-4oe70
systemd-private-cba9c77d21c6474391b8b8b746d3515e-polkit.service-myIs9W
systemd-private-cba9c77d21c6474391b8b8b746d3515e-power-profiles-daemon.service-9ne8cF
systemd-private-cba9c77d21c6474391b8b8b746d3515e-switcheroo-control.service-EjBChc
gry@gry-VMware-Virtual-Platform:~/scripts$ ls /nonexistent /tmp 1>out.txt 2>err.txt
gry@gry-VMware-Virtual-Platform:~/scripts$ cat out.txt
/tmp:
gdn3-config-err-KY4F8a
snap-private-tmp
systemd-private-cba9c77d21c6474391b8b8b746d3515e-colord.service-ixefik
systemd-private-cba9c77d21c6474391b8b8b746d3515e-fwupd.service-dLidwh
systemd-private-cba9c77d21c6474391b8b8b746d3515e-ModemManager.service-4oe70
systemd-private-cba9c77d21c6474391b8b8b746d3515e-polkit.service-myIs9W
systemd-private-cba9c77d21c6474391b8b8b746d3515e-power-profiles-daemon.service-9ne8cF
systemd-private-cba9c77d21c6474391b8b8b746d3515e-switcheroo-control.service-EjBChc
systemd-private-cba9c77d21c6474391b8b8b746d3515e-systemd-logind.service-o64hI8
systemd-private-cba9c77d21c6474391b8b8b746d3515e-systemd-oond.service-FZdUTH
systemd-private-cba9c77d21c6474391b8b8b746d3515e-systemd-resolved.service-15j05R
systemd-private-cba9c77d21c6474391b8b8b746d3515e-systemd-timesyncd.service-M1vKeN
systemd-private-cba9c77d21c6474391b8b8b746d3515e-upower.service-XzSXus
VMwareDnD
vmware-root_743-4257135038
gry@gry-VMware-Virtual-Platform:~/scripts$ cat err.txt
ls: 无法访问 '/nonexistent': 没有那个文件或目录

```

图 16: 实例 8: stdout 与 stderr 分开重定向

3.9 实例 9: \$? 与 &&、|| 条件执行

题目: 用 `echo $?` 查看上一条命令的退出码, 用 `true && echo ...` 与 `false || echo ...` 演示 && 与 || 的短路执行逻辑。

```
gry@gry-VMware-Virtual-Platform:~/scripts$ cat err.txt
ls: 无法访问 '/nonexistent': 没有那个文件或目录
gry@gry-VMware-Virtual-Platform:~/scripts$ echo $?
0
gry@gry-VMware-Virtual-Platform:~/scripts$ [ -d /tmp/mydir ] && echo 已存在
gry@gry-VMware-Virtual-Platform:~/scripts$ true && echo 成功
成功
gry@gry-VMware-Virtual-Platform:~/scripts$ false || echo 失败兜底
失败兜底
gry@gry-VMware-Virtual-Platform:~/scripts$
```

图 17: 实例 9: \$? 与短路执行的输出

3.10 实例 10: 带日期的备份文件

题目: 先创建 `notes.txt`, 再用 `cp notes.txt notes_$(date +%Y-%m-%d).txt`, 利用命令替换 `$(...)` 生成带当天日期 (2026-09-01) 的备份文件。

```
gry@gry-VMware-Virtual-Platform:~/scripts$ cd ~/scripts
gry@gry-VMware-Virtual-Platform:~/scripts$ echo 测试内容 > notes.txt
gry@gry-VMware-Virtual-Platform:~/scripts$ cp notes.txt notes_$(date +%Y-%m-%d).txt
gry@gry-VMware-Virtual-Platform:~/scripts$ ls -la notes*
-rw-rw-r-- 1 gry gry 13  9月  1 14:20 notes_2026-09-01.txt
-rw-rw-r-- 1 gry gry 13  9月  1 14:20 notes.txt
gry@gry-VMware-Virtual-Platform:~/scripts$
```

图 18: 实例 10: 带日期备份的运行结果

4 解题感悟

这一周之前我几乎没在命令行里做过事, 对 Git 和 LaTeX 也只是听过名字。做练习时我最大的感受就是“什么都要查、什么都会报错”——配置 SSH 时输错公钥、git 推送路径不对、合并分支时撞上冲突, 几乎没有一步是一次成功的。

但这些报错反而让我慢慢理解了每个命令在干什么。做完 4 个课上题加 10 个课后实例, 我现在能独立做几件事: 用 shell 命令管理文件、用管道处理文本、用 git 按题增量提交并解决一次合并冲突、用 LaTeX 编译出带交叉引用的 PDF。

我最大的收获不是一个具体命令, 而是学会了“报错先读错误信息、再一条条定位”。以前我一看报错就想放弃, 现在会从第一行猜问题在哪。虽然离熟练还很远, 但不再像一开始那样完全无从下手了。